Genie for External Users: Who It Is For and What Changes

Community reference, not official Databricks documentation. OAuth custom identity claims are announced on the Databricks *What's coming?* page; check your workspace with `sql/check_availability.sql`.

Your application's own users can ask a Genie Agent free-form questions about your data, and each of them sees only what their entitlements allow. None of them needs a Databricks account. Your backend authenticates as one service principal, stamps every request with the signed-in user's id inside an OAuth token that Databricks signs, and Unity Catalog filters every query by that id.

This guide explains who the pattern is for, what teams did before it, and what it changes. Choosing an Enforcement Model explains why the repo layers views and ABAC policies, the Implementation Guide walks through building it, the Admin Runbook covers operating it, and the Demo Runbook shows it end to end with two fictional companies.

Who this is for

You are	Typical users of your application
A team that runs a customer portal	members, participants, policyholders, patients, account holders
A software company or partner building on Databricks	your customers' staff, inside your product
A team serving partners or a network	brokers, advisors, franchisees, pharmacies, carriers, shippers, sellers
A team serving organizations with many sites	districts and schools, regions and stores

You'll recognize the need when you hear one of these:

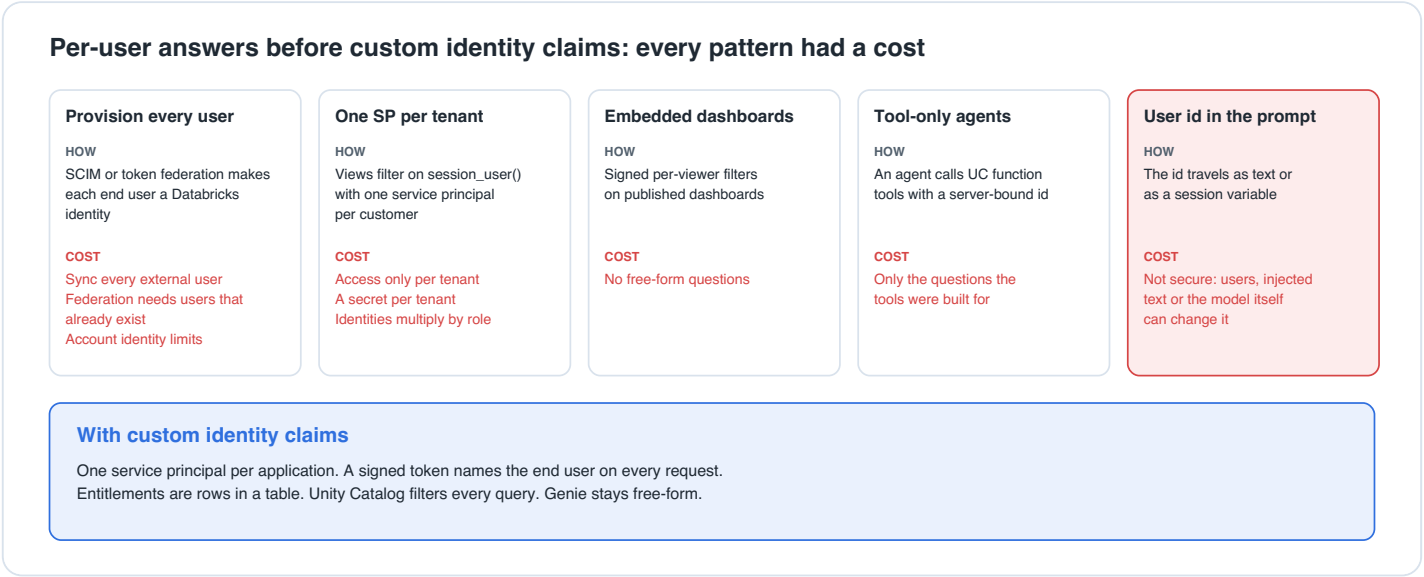
- "We want Genie, or 'ask your data', inside our portal or product."
- "Our users sign in through Auth0, Microsoft Entra External ID, Amazon Cognito, Okta or our own system, and we won't sync them to Databricks."
- "We have tens of thousands, or millions, of end users."
- "We have a service principal per customer and it is getting hard to manage."

- "Can we just pass the user id in the prompt?"
- "Our embedded dashboards work. Can users ask questions too?"
- "Some users have two roles, or should only see totals."

The problem: Genie answers as the caller

Genie writes SQL and runs it as the identity that called it. To give each person their own answer, Databricks has to know who that person is on every query, through something they can't change. For employees who already exist in Databricks, that is their own identity. For external users of your application, it has not been simple.

What teams did before



Five patterns teams used for per-user answers, and what each cost.

Pattern	How it works	What it costs
Provision every user	SCIM or token federation makes each end user a Databricks identity	You sync every external user. Token federation maps a token to a user who already exists; it doesn't create one. Large populations run into account identity limits.
One service principal per tenant	Views filter on `session_user()`; each customer gets its own service principal	Everyone in a tenant sees the same rows. Every tenant needs a secret. Finer access multiplies identities by role.

Pattern	How it works	What it costs
Embedded dashboards	Published dashboards filter per signed viewer	Viewers can't ask free-form questions.
Tool-only agents	An agent calls Unity Catalog function tools with a user id bound on the server	Users can only ask the questions the tools were built for.
User id in the prompt	The id travels as text or as a session variable	Not secure. The user, an injected instruction or the model itself can change text.

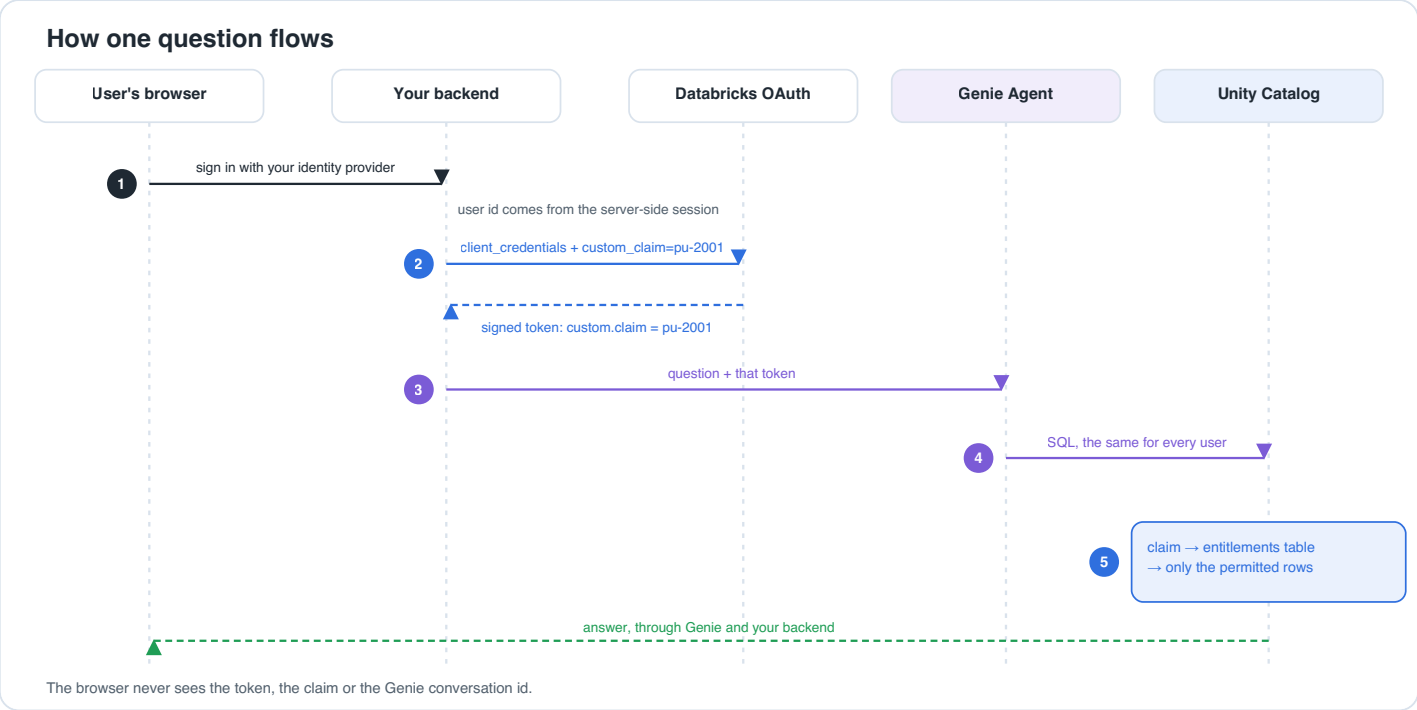
What changes with custom identity claims

	Before	With custom identity claims
Databricks identities for end users	One per user, or one service principal per tenant	One service principal per application
Scale ceiling	The account identity limit	Not tied to identity count. The claim is a string.
Access granularity	Per tenant	Per user, role, account or aggregate, in any combination
Onboarding a user	Create an identity, a secret and group memberships	Insert a row in your entitlements table
Revoking access	Delete the identity and clean up its secrets	Delete or expire the row. It applies on the next question.
A user with two roles	The app switches identities between questions	Two rows in a table
Secrets to manage	One per service principal, often thousands	One per application, or none with workload identity federation
Genie experience	Free-form, but only as granular as the identity	Free-form, per user
Where isolation is enforced	Unity Catalog, at identity level	Unity Catalog, at user level, from a signed token

Four things make it work:

- **One service principal per application.** It represents your backend. End users have no Databricks identity at all.
- **The claim travels inside a token Databricks signs.** Your backend reads the user's id from its own authenticated session (the chat and the browser never supply it) and asks Databricks for a short-lived token stamped with it. Tampered tokens are rejected.
- **Dynamic views compare rows to your entitlements table.** They read the claim with `current_oauth_custom_identity_claim()`. No matching grant means no rows.
- **Genie stays free-form.** Genie decides which SQL to write. Unity Catalog decides which rows come back.

How one question flows



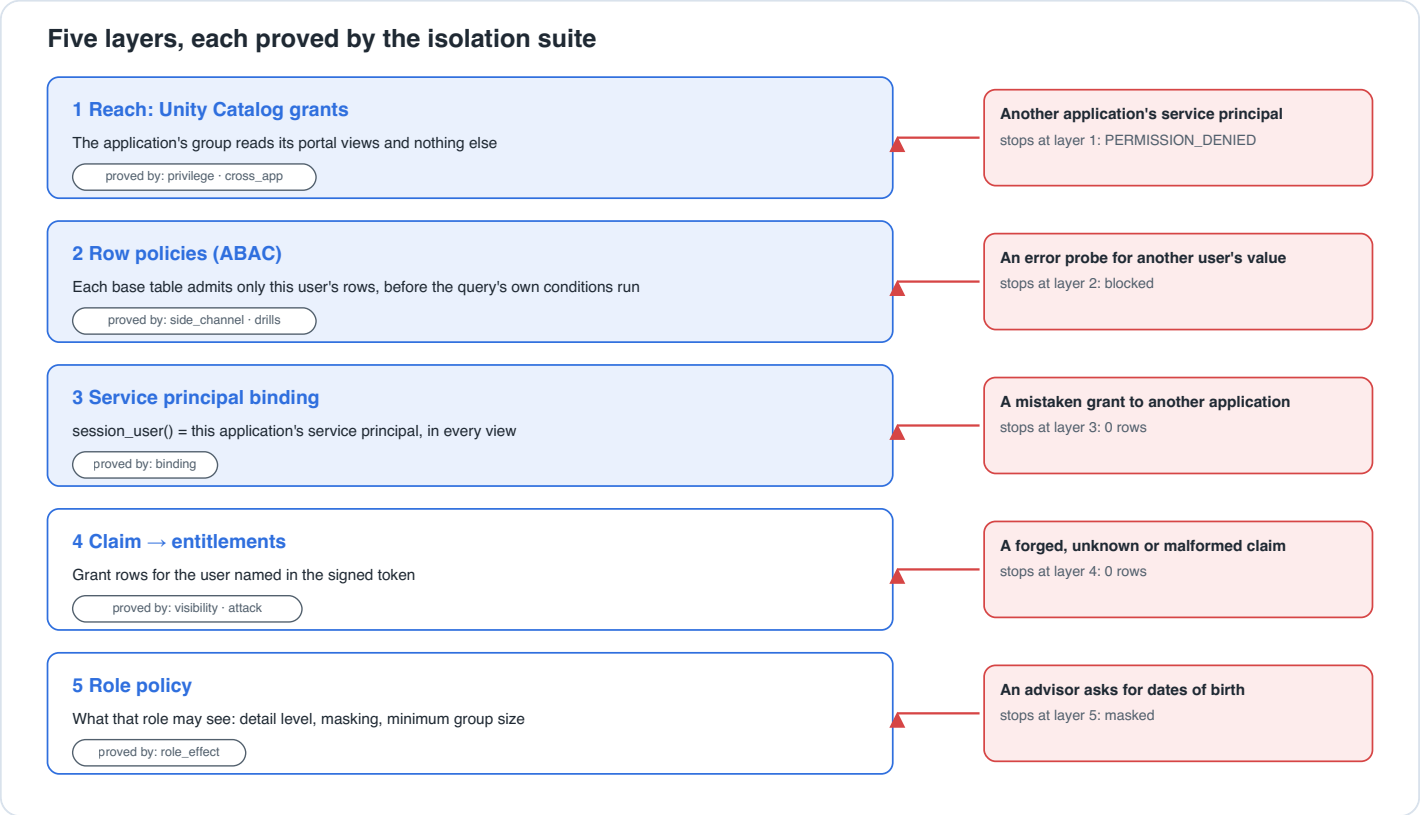
The five steps of one question. The browser never sees the token, the claim or the conversation id.

1. **Sign in.** The user signs in to your application with your identity provider.
2. **Stamp.** Your backend requests an OAuth token for its service principal and adds `custom_claim=<user id>`. Databricks returns a signed, short-lived token.
3. **Ask.** Your backend sends the question to the Genie Conversation API with that token.
4. **Write SQL.** Genie writes SQL. It is the same SQL for every user who asks the same question.
5. **Filter.** Unity Catalog evaluates the views and the row policies on the tables behind them. It reads the claim from the token, looks it up in your entitlements table and returns only the permitted rows.

According to the announcement, the Genie Conversation API, the SQL Statement Execution API and the Databricks JDBC driver honor the claim. You can use the same pattern from your own agent or service, without Genie.

> **Which Genie?** This pattern serves **Genie Agents through the Genie Conversation API**, called by your backend. Genie One, the assistant people use inside Databricks, runs as the person signed in to Databricks, so it isn't a path for your application's users.

Five layers of isolation



Each layer is checked separately by the isolation suite in this repo.

1. **Reach: Unity Catalog grants** decide which application can read which views. Each application's account group can read its own portal views and nothing else, so another application is denied.
2. **Row policies** on every base table those views read admit only the signed-in user's rows, and Unity Catalog applies them before any condition in the query runs. A user can't probe other users' rows through errors.
3. **Service principal binding** inside each view (`session_user()` must be this application's service principal) means that even a mistaken grant to another application returns no rows.
4. **Claim to entitlements** decides which end user. A forged, unknown or malformed claim matches no grant.
5. **Role policy** decides how much that user's role may see: their own records, aggregates only, masked columns, or a minimum group size.

Layers 1 and 2 cover each other's failure modes. A missing tag or a mis-targeted policy still leaves the views filtering by the claim, and a view written without its filter still sits on tables the policies protect. The misconfiguration drills in this repo make each mistake on a live deployment and show that every user still sees only their own rows.

Why views and policies together

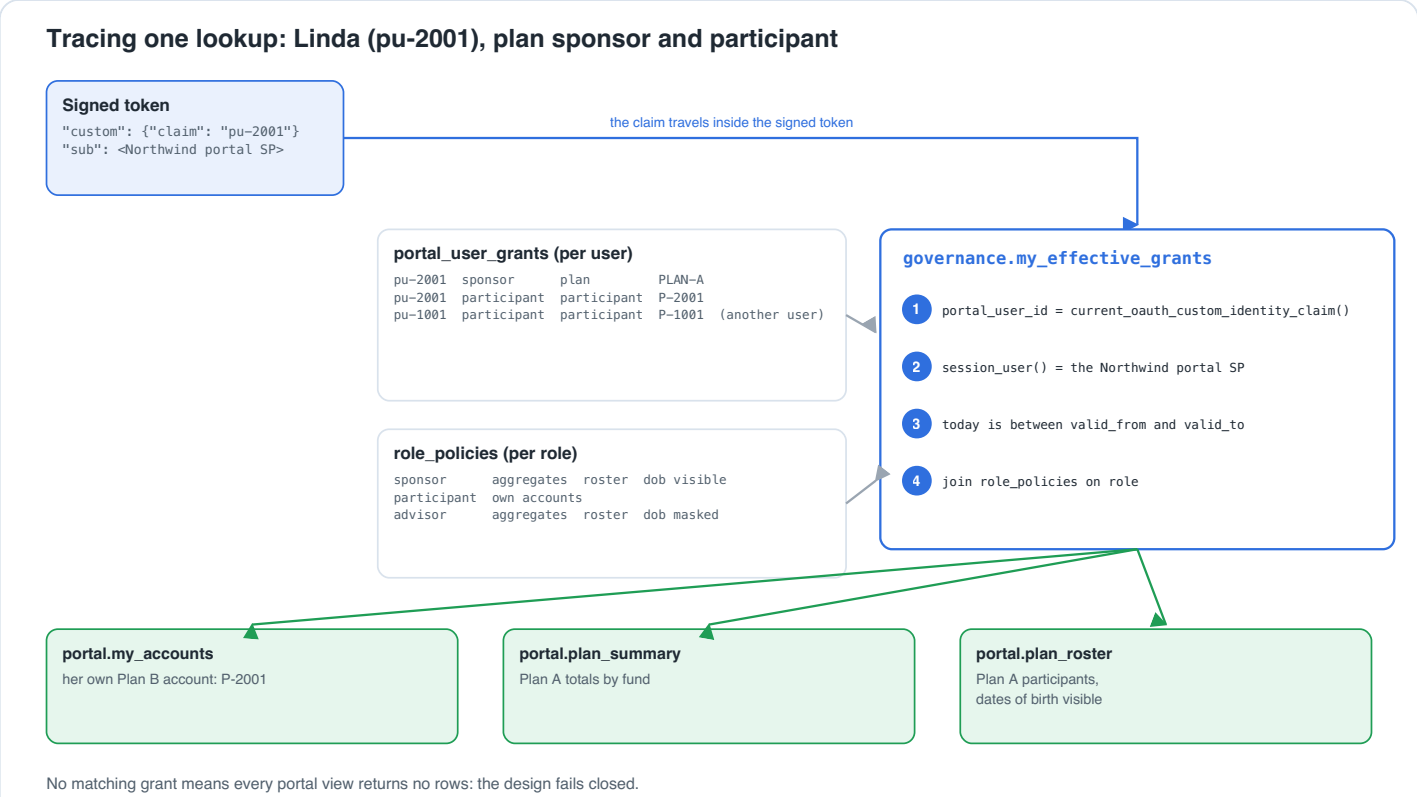
Claim-keyed views alone stay closed when someone forgets a step, but a user who can shape the SQL, as every Genie question does, can learn facts about other users' rows through errors. ABAC row filter policies alone block that, but an untagged table or column comes back unfiltered. Layering them gives you both properties. Choosing an Enforcement Model has the failure matrix, the live probe results and the performance numbers, and when a single layer is enough.

Entitlements are data: per user and per role

Two small tables hold every rule:

- **Per-user grants**: one row per user, role and scope, with optional start and end dates. Onboarding is an insert. Revoking is a delete or an end date. A user with two roles has two rows.
- **Per-role policies**: one row per role, saying what that role may see. Change a row and every user with that role changes on their next question.

One governance view resolves the claim against both, and every portal view joins to it. That keeps the security logic in a single definition you can review.



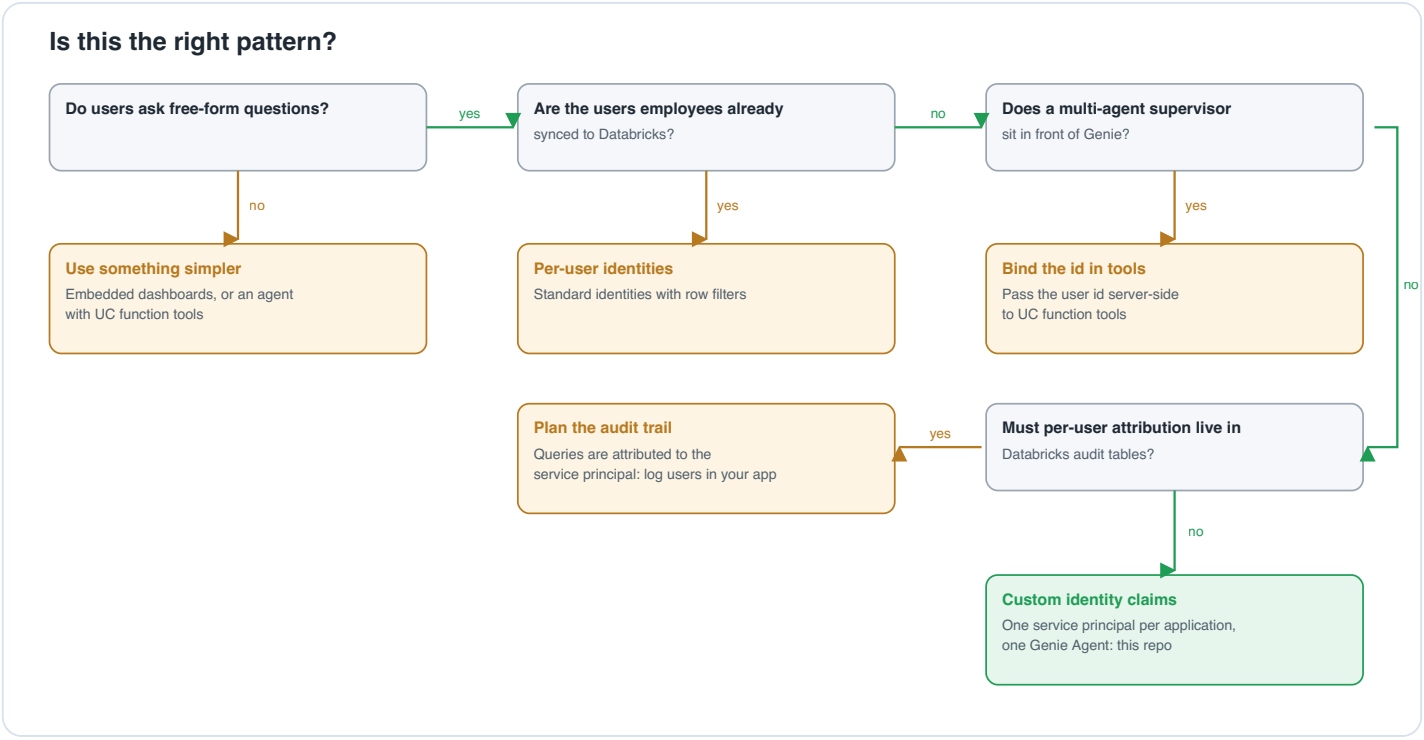
One lookup traced end to end for a user who holds two roles.

The entitlement pattern catalog shows nine patterns built on these two tables, each traced on a live workspace.

The flexibility you get

- **Onboard in a row.** No identity, secret or group membership to create.
- **Revoke on the next question.** Delete the row, or give it an end date.
- **Several roles per person.** A plan sponsor who is also a participant gets two rows and sees both.
- **Totals only.** Aggregate views with a minimum group size per role.
- **Mask by role.** The same view hides a column for one role and shows it for another.
- **Time-bound access.** A consultant's engagement ends on a date; access ends with it.
- **Rules from your data.** A parent sees a minor child's claims, computed from dates of birth, with no extra rows.
- **Many applications.** Each application has its own service principal and its own views, and one can't read another's.
- **Any identity provider.** Your sign-in stays exactly as it is.

Is this the right pattern?



A quick way to decide.

Situation	A better fit
Only fixed, known questions	Embedded dashboards, or an agent with Unity Catalog function tools
Users are employees already synced to Databricks	Per-user identities with row filters and column masks
A multi-agent supervisor sits in front of Genie	Bind the user id on the server in Unity Catalog function tools
Per-user attribution must live inside Databricks audit tables	Plan for an application audit log: query history attributes queries to the service principal

Availability

OAuth custom identity claims are announced on the Databricks *What's coming?* page (AWS, Azure). There is no setting to turn on. Check your own workspace in the SQL editor:

```
DESCRIBE FUNCTION current_oauth_custom_identity_claim;
-- A description comes back: available.   [UNRESOLVED_ROUTINE]: not available in this workspace
yet.

SELECT current_oauth_custom_identity_claim() AS claim;
-- NULL, or [OAUTH_CUSTOM_IDENTITY_CLAIM_NOT_PROVIDED]: available. Your editor session simply has
no claim.
```

The third check is a test token request with `custom_claim` set. The Implementation Guide has the commands, and the bundle in this repo runs all three checks before it deploys anything.

FAQ

Does Databricks learn who my users are? Only the opaque id string you choose. Databricks creates no names, emails or accounts for them.

What stops the AI from returning someone else's data? The AI never decides whose data it is. The id is inside a signed token, and Unity Catalog filters every query by it. A prompt that claims to be someone else changes nothing.

Does this replace our login? No. Your sign-in stays as it is. The claim travels from your backend to Databricks after the user signs in.

How is this different from token federation? Token federation signs in a user who already exists in Databricks. With custom identity claims, the end user never has to exist in Databricks.

Can one person hold several roles? Yes. Each role is its own grant row, and the views combine them. When grants disagree on masking, the least restrictive grant applies.

What happens when a user has no grants? Every view returns no rows, and Genie says the account has no access to that information.

Can we use this without Genie? Yes. The Statement Execution API and JDBC honor the claim too, so your own services and agents can query the same views.

Can the claim carry roles or a list of plans? The claim is one string. Keep it an opaque user id and keep roles and scopes in the entitlement tables: a change then applies on the user's next question without reissuing tokens, and every rule lives in one place you can audit.

Does it scale to millions of users and thousands of tenants? Users and tenants are rows in your tables, so onboarding stays an insert. Cluster the protected tables on the columns the policies filter by (tenant, member). The entitlement lookup then prunes files, so a scoped user reads only the files that hold their rows.

Does this work with Genie One? No. Use a Genie Agent through the Genie Conversation API (see "Which Genie?" above).